

- JS2-数据类型
 - 1 数据类型种类
 - 1.1 基本类型
 - 1.2 引用类型
 - 2 数据类型检测
 - 2.1 typeof
 - 2.2 instanceof
 - 2.3 Object.prototype.toString
 - 2.4 Array.isArray()
 - 2.5 总结

JS2-数据类型

1 数据类型种类

在JavaScript中，数据类型分为**基本类型**和**引用类型**。根据最新的ECMAScript标准，JavaScript共有**8种数据类型**。

1.1 基本类型

基本数据类型是存储在栈中的简单值，按值传递，主要包括以下7种：

1. **Number**：表示数字，包括整数和浮点数，例如42或 3.14。（Infinity，NaN）
2. **String**：表示文本值，例如"Hello"，"你好"。
3. **Boolean**：布尔值，只有true，false。
4. **Undefined**：表示变量已声明但未赋值。
5. **Null**：表示空值。
6. **Symbol**：ES6引入的一种唯一且不可变的基本数据类型，用于解决属性名冲突。
7. **BigInt**：ES6引入的一种用于表示任意精度的大整数，超出Number的安全范围。

考点：**null** 与 **undefined** 的区别

特性	undefined	null
语义	未定义	空值
数据类型	"undefined"	"object" (bug)

特性	undefined	null
自动发生	变量声明未赋值	通常人为赋值

```
console.log(undefined == null) // true
console.log(undefined === null) // false
```

1.2 引用类型

引用类型（Object）存储在堆中，按引用传递，常见引用类型主要包括：

1. **Object**：表示复杂的数据结构，例如对象、数组、函数等。
2. **Array**：对象的一种特殊形式，用于存储有序集合。
3. **Function**：一种特殊的对象，用于封装可执行代码。

考点：基础概念

以下哪一个不是JavaScript的数据类型：

A. String B. Boolean C. Integer D. Undefined

考察的是记忆JS的8种数据类型，并辨识出 **Integer** 这类其他语言的概念

2 数据类型检测

2.1 typeof

typeof 运算符用于返回操作数的类型。它返回一个**字符串**，表示操作数的类型。

```
// 数值
console.log(typeof 37); // "number"
console.log(typeof 3.14); // "number"
console.log(typeof NaN); // "number"
console.log(typeof Infinity); // "number"
// 字符串
console.log(typeof "hello"); // "string"
console.log(typeof 'template literal'); // "string"
// 布尔值
console.log(typeof true); // "boolean"
console.log(typeof false); // "boolean"
```

```
// 符号
console.log(typeof Symbol()); // "symbol"
// 未定义
console.log(typeof undefined); // "undefined"
console.log(typeof undeclaredVariable); // "undefined"
// 大整数
console.log(typeof 10n); // "bigint"
```

typeof的局限性：

1. typeof null 的结果是 "object"。这是JS著名的历史遗留Bug，为了兼容性而保留了下来。

```
console.log(typeof null); // "object"
```

2. typeof 无法区分引用类型（函数除外）。

```
// 对象
console.log(typeof { a: 1 }); // "object"
console.log(typeof [1, 2, 3]); // "object"
console.log(typeof new Date()); // "object"
```

3. typeof 会对函数区分对待，并返回 "function"。这也是来自于 JavaScript 语言早期的问题。从技术上讲，这种行为是不正确的，但在实际编程中却非常方便。

```
// 函数
console.log(typeof function() {}); // "function"
console.log(typeof class C {}); // "function"
```

考点：

1. typeof 的输出范围

以下不属于typeof运算符返回值的是？ A. "string" B. "function" C. "object" D. "null"

正确答案是D。

因为 typeof 的返回值只有8

种："undefined"、"boolean"、"number"、"string"、"bigint"、"symbol"

、"function"和"object"

2. typeof 的判断限制

可以用typeof来判断的基本类型有？ A. undefined B. null C. array D. object

答案是A。这题考的是 **typeof** 的局限性

2.2 instanceof

instanceof运算符用于通过查找原型链来检测某个变量是否为某个类型数据的实例。

instanceof只能正确判断引用数据类型。

```
const a = [0, 1, 2];
console.log(a instanceof Array); // true
console.log(a instanceof Object); // true

const b = {name: 'xx'};
console.log(b instanceof Array); // false
console.log(b instanceof Object); // true
```

有一点需要留意，a 同时还隶属于 Object 类。因为从原型上来讲，Array 是继承自 Object 的。instanceof 在检查中会将原型链考虑在内。因此我们在判断一个变量时数组还是对象时，应该先判断数组类型，然后再去判断对象类型。如果先判断对象，那么数组值也会被判断为对象类型，这无法满足要求。

2.3 Object.prototype.toString

每种引用类型都会直接或者间接继承自Object类型，因此它们都包含toString()函数。不同数据类型的toString()类型返回值也不一样，所以通过toString()函数可以准确判断值的类型。

```
// 基本类型
console.log(a.call(2)); // [object Number]
console.log(a.call(true)); // [object Boolean]
console.log(a.call("str")); // [object String]
console.log(a.call(undefined)); // [object Undefined]
console.log(a.call(null)); // [object Null]
console.log(a.call(Symbol())); // [object Symbol]
console.log(a.call(10n)); // [object BigInt]
// 引用类型
console.log(a.call([])); // [object Array]
```

```
console.log(a.call(function () {})); // [object Function]
console.log(a.call({})); // [object Object]
```

2.4 Array.isArray()

JavaScript 中的一个静态方法，用于确定传递的值是否是一个数组。

```
// 下面的函数调用都返回 true
Array.isArray([]); // true
Array.isArray([1]); // true
Array.isArray(new Array()); // true
Array.isArray(new Array("a", "b", "c", "d")); // true
Array.isArray(new Array(3)); // true
Array.isArray(Array.prototype); // true
```

2.5 总结

方法	适用场景	返回值
typeof	原始数据类型	string
instanceof	引用数据类型	true/false
Object.prototype.toString	最精确的类型判断	string
Array.isArray()	判断数组	true/false